📜 AI-BASED SCHEMA GENERATOR USING DOCKER

Title: AI-Based Schema Generator using Docker

2. Problem Statement

The objective is to develop an AI-based script using Docker that can automatically design a database schema based on user requirements. This system helps in efficient data management by generating structured schemas dynamically.

---

3. Objective

- To create an AI-based schema generator
- To deploy the application using Docker
- To automate database schema creation
- To simplify data management

---

4. System Requirements

- Python (Anaconda or Python 3.x)
- Docker Desktop
- VS Code / Terminal
- Internet connection

---

5. Project Architecture

- User inputs requirements (fields, data types)
- AI script processes input
- Generates schema (SQL format)
- Docker container runs the application

---

6. Implementation Steps

Step 1: Create Python Script

```
def generate_schema(table_name, columns):
    schema = f"CREATE TABLE {table_name} (\n"
    for col, dtype in columns.items():
        schema += f"  {col} {dtype},\n"
    schema = schema.rstrip(",\n") + "\n);"
    return schema

# Example input
```

```
table_name = "employee"
columns = {
   "empId": "INT",
   "employeeAge": "INT",
   "salary": "FLOAT",
   "country": "VARCHAR(50)"
}

print(generate_schema(table_name, columns))
```

Description: This script generates SQL schema based on user input.

Step 2: Create Dockerfile

```
FROM python:3.9
WORKDIR /app
COPY . /app
CMD ["python", "schema.py"]
```

Description: Defines container environment.

Step 3: Build Docker Image

```
docker build -t schema-generator .
```

Description: Builds Docker image.

Step 4: Run Docker Container

```
docker run schema-generator
```

Description: Executes schema generator.

---

7. Output

Example Generated Schema:

```
CREATE TABLE employee (
 empId INT,
 employeeAge INT,
 salary FLOAT,
 country VARCHAR(50)
);
```

---

8. Applications

- Database design automation
- Data management systems

- AI-assisted development
- Enterprise data solutions

---

9. Advantages

- Saves time in schema design
- Reduces manual errors
- Easy deployment using Docker
- Scalable and reusable

---

10. Conclusion

- Successfully developed an AI-based schema generator
- Implemented using Python and Docker
- Automated database schema creation
- Useful for real-world data management applications
